Plan 4 — Grid-Formation

Formation mission · domain `grid-formation-drone-mission` · plan kind `grid_formation`
A complete walkthrough of domain.pddl, problem.pddl and every Python function that runs them.

1. What this plan does

Grid-Formation is picked exactly like V-Formation - a **source**, a **destination**, and optionally a **restricted area** - but the swarm flies as a rectangular block rather than a V. One drone is the **leader**, sitting in the front row; every other drone holds a fixed cell in a grid behind and beside it, 10 metres apart in every direction.

As in the V, **ENHSP plans only the leader's corridor**. Every other drone's track is derived geometrically from the leader's flown path afterwards. The difference is that a V's wings come in symmetric pairs, so two calls suffice, whereas a grid's members each sit at their own distinct row-and-column offset and need one call apiece.

1.1 The grid's geometry

A 2x4 grid of 8 drones

```
                    direction of travel
                            ^
                            |

 row 0   [ drone-r0c0 ] [ drone-lead ] [ drone-r0c2 ] [ drone-r0c3 ]
               |              |              |              |
   10 m        |              |              |              |
               |              |              |              |
 row 1   [ drone-r1c0 ] [ drone-r1c1 ] [ drone-r1c2 ] [ drone-r1c3 ]

          col 0          col 1          col 2          col 3
          <---- 10 m ---><---- 10 m ---><---- 10 m ---->

GRID_SPACING_M = 10.0 m, front-to-back and side-to-side alike.

The leader sits in the FRONT row, in the column nearest the middle:
    leader_cell = (0, (cols - 1) // 2)

Each drone's offsets, relative to the leader:
    slot-along-offset = -row * 10       (negative = behind)
    slot-cross-offset = (col - leader_col) * 10   (negative = left)

Each drone coordinates with only its up-to-4 immediate neighbours
(front, back, left, right) - never the whole swarm, and never diagonals.
```

1.2 How the drone count becomes a shape

Drones	Shape	Why
4	2 x 2	perfect square
6	2 x 3	largest divisor at or below sqrt(6) is 2
8	2 x 4	largest divisor at or below sqrt(8) is 2
9	3 x 3	perfect square
12	3 x 4	largest divisor at or below sqrt(12) is 3
16	4 x 4	perfect square
7 (prime)	1 x 7	no divisor but 1 - degenerates to a single line

A perfect-square count forms a square; anything else forms the most-square rectangle that tiles it **exactly**, so no cell is ever left empty. The minimum is `MIN_GRID_DRONES = 4` - the smallest count that reads as a grid rather than a line.

1.3 The pipeline, end to end

The Grid-Formation planning pipeline

```
GUI: operator picks source + destination (+ optional no-fly polygon)
     and ticks N drones (N >= 4)
                  |
                  v
services/plan_service.py   run_async(...) -> _Worker.run(...) -> _run(...)
     plan_kind() == "grid_formation"  ->  _run_grid_formation(...)
                  |
                  +--> grid_dimensions(N)  ->  (rows, cols)
                  |
                  +--> expand_grid_drones(template, N)
                  |       the 1x2 template -> the full rows x cols grid,
                  |       every cell's offsets, and a rebuilt neighbour mesh
                  |
                  +--> retarget_grid_problem(..., no_fly_buffer_m=
                  |         NO_FLY_BUFFER_M + GRID_SPACING_M * lateral_reach)
                  v
     plans/gridformation/runs/<timestamp>/problem.pddl
                  |
                  v
engine/pddl_planner.py  run_enhsp(..., extra_args=["-gro", "naive"],
                                  timeout_s = max(30, rows * cols))
     -> extract_formation_corridor(steps)   ->  ONE corridor
                  |
                  v
formation_member_route(lead_route, along_m=row*10,
                       cross_m=(col - leader_col)*10)
     called ONCE PER CELL
                  |
                  v
PlanRunResult(per_drone_waypoints = one track per grid cell)
```

2. domain.pddl explained

The grid domain is closely parallel to the V's - same phases, same collective-action pattern, same neighbour model. This chapter covers all 14 actions, noting at each point where the grid differs from the V.

2.1 Header, requirements and types

domain.pddl - lines 1-19

```
(define (domain grid-formation-drone-mission)

    (:requirements
        :strips :typing :equality :numeric-fluents
        :negative-preconditions :disjunctive-preconditions
        :universal-preconditions :conditional-effects)

    (:types
        drone
        location
        controller
    ))
```

The same requirement set as the V, and for the same reasons: `:universal-preconditions` for the `forall` constructs that make the domain generic over the drone count, and `:conditional-effects` for the `when` clauses in `close-up-grid`.

Three types, not four. The V declares a `direction` type for its eight compass objects; the grid does not. A grid member's position is read off its numeric `slot-along-offset`/`slot-cross-offset` directly rather than through a compass label, so there is nothing for a `direction` type to name - and correspondingly nothing for `retarget_grid_problem` to recompute after a move.

2.2 Predicates

Group A - position and flight state

```
;; Drone position, and the formation's own position
(at ?d - drone ?l - location)
(formation-at ?l - location)

;; Mission locations and routes
(source ?l - location) (destination ?l - location)
(connected ?from - location ?to - location)
(safe-route ?from - location ?to - location)

;; Flight state
(landed ?d - drone) (airborne ?d - drone)
(flying ?d - drone) (at-cruise-altitude ?d - drone)
```

Identical to the V's. `formation-at` again tracks the formation as a whole, separately from any individual drone, which is what produces the leader-moves-then-members-follow rhythm.

Group B - slots and formation status

```
;; Formation assignment
(has-slot ?d - drone)
(grid-member ?d - drone)
(formation-leader ?d - drone)
(in-formation ?d - drone)
(grid-formation-established)
(grid-closed)
```

`(grid-member ?d)` plays exactly the role `wing-drone` plays in the V: it is the fact every `forall` quantifies over, written as 'either this drone is not a grid member, or it satisfies ...'. The domain therefore works unchanged for any grid size.

There is no `slot-apex` equivalent. The V declares both `formation-leader` and `slot-apex` for its lead drone and checks both; the grid declares only `formation-leader`. The apex of a V is a geometrically special position - the point of the V - whereas a grid leader is simply one cell among many that happens to be designated.

`(grid-closed)` is the interlock, matching the V's `wings-closed`: `formation-cruise` clears it, `close-up-grid` restores it, and `formation-cruise` requires it.

Group C - the neighbour model

```
;; Neighbor communication
(grid-neighbor ?d1 - drone ?d2 - drone)
(neighbor-comm-ok ?d - drone)
```

The drone-to-drone model, and the one the V's was later copied from. Chapter 5 covers it in full.

`grid-neighbor` is pairwise and **descriptive** - it records front/back and left/right adjacency, with no diagonals. No action reads it.

`neighbor-comm-ok` is per-drone and is what the actions actually check. The domain's own comment, quoted in `expand_grid_drones`'s docstring, gives the reason for the split: a per-drone flag keeps each action's cost linear in the drone count, whereas checking every pair would be quadratic - which matters far more here than in a V, since a grid has roughly twice as many adjacencies as drones.

Group D - health, checks and mission status

```
;; Pre-flight status
(preflight-done ?d - drone)

;; System health
(gps-ok ?d - drone) (communication-ok ?d - drone) (drone-healthy ?d - drone)

;; Failure states
(gps-lost ?d - drone) (communication-lost ?d - drone)
(health-failure ?d - drone) (controller-notified ?d - drone)

;; Mission status
(mission-completed ?d - drone)
(formation-mission-complete)
```

Identical to the V's group D, including the nullary `(formation-mission-complete)` set by the same action that marks every drone complete.

2.3 Functions

domain.pddl - lines 75-106

```
(:functions
    (battery ?d - drone) (max-battery ?d - drone)
    (minimum-return-battery ?d - drone)

    (altitude ?d - drone) (hover-altitude ?d - drone) (target-altitude ?d - drone)
    (climb-energy ?d - drone) (cruise-leg-energy ?d - drone)
    (energy-required ?from - location ?to - location)
    (distance ?from - location ?to - location)

    (health ?d - drone) (minimum-health ?d - drone)
    (latitude ?l - location) (longitude ?l - location)

    ;; Grid slot offsets
    (slot-along-offset ?d - drone)
    (slot-cross-offset ?d - drone)

    ;; Staggered launch timing
    (seconds-since-leader-airborne)
    (grid-launch-delay))
```

The same fluent set as the V, with `wing-launch-delay` renamed `grid-launch-delay`.

The two slot offsets are again **descriptive bookkeeping**: `expand_grid_drones` computes and writes them for every cell, but the actual flown tracks are produced by `formation_member_route` from `GRID_SPACING_M` directly. Unlike the V - where `FORMATION_BACK_M` and `FORMATION_SIDE_M` in `plan_service.py` duplicate numbers that also appear in the problem file - the grid derives both representations from the single constant `GRID_SPACING_M`, so there is nothing to keep in sync by hand.

Note also that the two representations use **opposite sign conventions** for 'behind', which is called out explicitly in `_run_grid_formation`. See section 4.5.

2.4 The 14 actions

Phase 1 - getting airborne

ACTION 1 of 14 - pre-flight-check

```
(:action pre-flight-check
    :parameters (?d - drone ?l - location)
    :precondition (and (at ?d ?l) (source ?l) (landed ?d) (has-slot ?d)
                       (gps-ok ?d) (communication-ok ?d) (drone-healthy ?d)
                       (>= (health ?d) (minimum-health ?d))
                       (> (battery ?d) (minimum-return-battery ?d)))
    :effect (preflight-done ?d))
```

Identical to the V's. Every drone must be landed at the source, hold a slot, be fully healthy, and carry more than its return reserve before it may take off.

ACTION 2 of 14 - takeoff [the staggered launch]

```
(:action takeoff
    :parameters (?d - drone ?l - location)
    :precondition (and (at ?d ?l) (source ?l) (landed ?d) (preflight-done ?d)
                       (gps-ok ?d) (communication-ok ?d) (drone-healthy ?d)
                       (or (formation-leader ?d)
                           (>= (seconds-since-leader-airborne) (grid-launch-delay))))
    :effect (and (not (landed ?d)) (airborne ?d) (flying ?d)
                 (assign (altitude ?d) (hover-altitude ?d))))
```

The same disjunctive precondition as the V: the leader may go immediately, every other drone must wait until the leader has been airborne for `grid-launch-delay` seconds.

This matters more for a grid than for a V, simply because there are more of them. A 16 drone grid launching simultaneously from one point would put sixteen aircraft in the same volume of air at the same moment.

`(assign (altitude ?d) (hover-altitude ?d))` sets the altitude outright to the 2 m hover height. This fact is the one the 2026-09-16 defect corrupted - see chapter 6.

ACTION 4 of 14 - hold-position [the clock]

```
(:action hold-position
    :parameters (?lead - drone)
    :precondition (and (formation-leader ?lead) (airborne ?lead)
                       (< (seconds-since-leader-airborne) (grid-launch-delay))
                       (gps-ok ?lead) (communication-ok ?lead) (drone-healthy ?lead))
    :effect (increase (seconds-since-leader-airborne) 1))
```

The clock, advanced one second per application, applicable only while it is still below the delay. With the template's delay of 5 the planner must apply it exactly five times, which is plainly visible at steps 3 to 7 of the worked run in chapter 6. Only the leader can do it.

ACTION 3 of 14 - climb-to-cruise-altitude

```
(:action climb-to-cruise-altitude
    :parameters (?d - drone)
    :precondition (and (airborne ?d) (not (at-cruise-altitude ?d))
                       (< (altitude ?d) (target-altitude ?d))
                       (gps-ok ?d) (communication-ok ?d) (drone-healthy ?d)
                       (> (battery ?d) (climb-energy ?d)))
    :effect (and (assign (altitude ?d) (target-altitude ?d))
                 (at-cruise-altitude ?d)
                 (decrease (battery ?d) (climb-energy ?d))))
```

Takes each drone from its 2 m hover height to the 50 m cruise altitude in one step, billing 4 units of `climb-energy`. The flag plus the `not` precondition make it fire at most once per drone.

This action is where the 2026-09-16 defect actually surfaced. Its precondition `(< (altitude ?d) (target-altitude ?d))` compares two fluents - and when `hover-altitude` was left undefined for a cloned drone, `takeoff`'s assignment had no value to copy, leaving `altitude` undefined too. ENHSP's preprocessing flagged the resulting problem as unsolvable.

Phase 2 - forming up

ACTION 5 of 14 - take-grid-slot

```
(:action take-grid-slot
    :parameters (?d - drone)
    :precondition (and (airborne ?d) (has-slot ?d) (at-cruise-altitude ?d)
                       (>= (altitude ?d) (target-altitude ?d))
                       (not (in-formation ?d))
                       (gps-ok ?d) (communication-ok ?d) (drone-healthy ?d))
    :effect (in-formation ?d))
```

Each drone individually moves into its assigned cell. Purely individual - it says nothing about the other drones - and fires at most once per drone thanks to the `(not (in-formation ?d))` precondition. The collective check comes next.

ACTION 6 of 14 - establish-grid-formation [the collective gate]

```
(:action establish-grid-formation
    :parameters (?lead - drone ?l - location)
    :precondition (and (formation-leader ?lead) (in-formation ?lead) (at ?lead ?l)
                       (at-cruise-altitude ?lead)
                       (>= (altitude ?lead) (target-altitude ?lead))
                       (neighbor-comm-ok ?lead)

                       (forall (?m - drone)
                           (or (not (grid-member ?m))
                               (and (in-formation ?m)
                                    (at ?m ?l)
                                    (at-cruise-altitude ?m)
                                    (>= (altitude ?m) (target-altitude ?m))
                                    (neighbor-comm-ok ?m)))))
    :effect (and (grid-formation-established) (formation-at ?l) (grid-closed)))
```

Fires only when the leader **and every grid member** are in formation, at the same location, at cruise altitude, and have healthy neighbour links.

How the `forall` works. The body reads 'either `?m` is not a grid member, or it satisfies all of these' - the standard PDDL idiom for restricting a universal quantifier to a subset. Drones without the `grid-member` fact, which is just the leader, satisfy the first disjunct trivially. This is what makes the domain indifferent to whether the grid is 2x2 or 6x6.

`(at ?m ?l)` requires every member at the same location object as the leader. In PDDL the entire formation occupies one location; the real 10-metre spacing lives entirely in the Python geometry layer. Without this abstraction the planner would have to reason about dozens of distinct positions and the problem would not be tractable.

The only difference from the V's equivalent is the absence of the `slot-apex` check, since the grid has no apex predicate.

Phase 3 - cruising as a formation

ACTION 7 of 14 - formation-cruise [the only action the Python side reads]

```
(:action formation-cruise
    :parameters (?lead - drone ?from - location ?to - location)
    :precondition (and (grid-formation-established) (grid-closed)
                       (formation-leader ?lead) (in-formation ?lead)
                       (formation-at ?from) (at ?lead ?from)
                       (connected ?from ?to) (safe-route ?from ?to)
                       (at-cruise-altitude ?lead)
                       (>= (altitude ?lead) (target-altitude ?lead))
                       (gps-ok ?lead) (communication-ok ?lead) (drone-healthy ?lead)
                       (neighbor-comm-ok ?lead)
                       (> (battery ?lead) (energy-required ?from ?to)))
    :effect (and (not (formation-at ?from)) (formation-at ?to)
                 (not (at ?lead ?from)) (at ?lead ?to)
                 (not (grid-closed))
                 (decrease (battery ?lead) (energy-required ?from ?to))))
```

The leader flies one leg and the formation marker moves with it. This is the action `extract_formation_corridor` looks for - and note that it has the **same name** as the V's, which is exactly why that one extractor serves both plans unchanged.

Only the leader moves; the members stay put and catch up in the next action. **Only the leader's battery is billed**; each member pays its own `cruise-leg-energy` in `close-up-grid`.

`(not (grid-closed))` is the interlock. Since `grid-closed` is also a precondition, the leader cannot fly a second leg until `close-up-grid` has restored it. Leg, close up, leg, close up - the grid can never string itself out across a detour.

ACTION 8 of 14 - close-up-grid [the most intricate action in the project]

```
(:action close-up-grid
    :parameters (?lead - drone ?to - location)
    :precondition (and (grid-formation-established) (not (grid-closed))
                       (formation-leader ?lead) (in-formation ?lead)
                       (formation-at ?to) (at ?lead ?to)
                       (forall (?m - drone)
                           (or (not (grid-member ?m))
                               (and (in-formation ?m) (at-cruise-altitude ?m)
                                    (>= (altitude ?m) (target-altitude ?m))
                                    (gps-ok ?m) (communication-ok ?m) (drone-healthy ?m)
                                    (neighbor-comm-ok ?m)
                                    (> (battery ?m) (cruise-leg-energy ?m))))))
    :effect (and
        (forall (?m - drone) (forall (?x - location)
            (when (and (grid-member ?m) (at ?m ?x))
                  (not (at ?m ?x)))))
        (forall (?m - drone)
            (when (grid-member ?m) (at ?m ?to)))
        (forall (?m - drone)
            (when (grid-member ?m)
                  (decrease (battery ?m) (cruise-leg-energy ?m))))
        (grid-closed)))
```

Moves **every** grid member to the leader's new location in a single action and bills each one for the leg. Its effect is the one genuinely advanced PDDL construction in the codebase and repays careful reading.

The first effect is a doubly-nested conditional delete. For every member `?m` and every location `?x`, if `?m` is at `?x`, delete that fact. The inner `forall` over **locations** is unavoidable: PDDL has no way to express 'delete wherever it happens to be', so the effect must range over every possible location and delete conditionally.

The second adds them all at the new location, and the third bills each battery. Three separate blocks because each does a different kind of thing - delete, add, decrease.

Every member's battery is checked before any of them move. This is all-or-nothing: the grid does not partially close up. If one member is too low, the action is inapplicable and the formation is stuck - a correct model of a rigid formation, which can only go as far as its weakest member.

And every member's `neighbor-comm-ok` is checked. One member that has lost contact with its neighbours blocks the entire grid from advancing.

ACTION 9 of 14 - complete-grid-mission

```
(:action complete-grid-mission
    :parameters (?lead - drone ?dest - location)
    :precondition (and (destination ?dest) (grid-formation-established)
                       (formation-at ?dest)
                       (formation-leader ?lead) (in-formation ?lead) (at ?lead ?dest)
                       (forall (?m - drone)
                           (or (not (grid-member ?m))
                               (and (in-formation ?m) (at ?m ?dest)))))
    :effect (and (mission-completed ?lead)
                 (forall (?m - drone)
                     (when (grid-member ?m) (mission-completed ?m)))
                 (formation-mission-complete)))
```

Closes the mission once the entire grid is at the destination and still in formation. One action marks the leader complete explicitly and every member complete through a conditional effect, then sets the nullary mission flag - satisfying all the goal's per-drone `mission-completed` conjuncts in a single step, however many drones there are.

Phase 4 - fault detection

Four detectors, all requiring `(flying ?d)`, all setting a failure fact, all clearing both `flying` and - crucially - `in-formation`, which ejects the faulty drone and thereby blocks every collective action above.

ACTIONS 10, 11 and 13 of 14 - the three inherited detectors

```
(:action detect-gps-loss
    :parameters (?d - drone)
    :precondition (and (flying ?d) (not (gps-ok ?d)))
    :effect (and (gps-lost ?d) (not (flying ?d)) (not (in-formation ?d))))

(:action detect-communication-loss
    :parameters (?d - drone)
    :precondition (and (flying ?d) (not (communication-ok ?d)))
    :effect (and (communication-lost ?d) (not (flying ?d)) (not (in-formation ?d))))

(:action detect-health-failure
    :parameters (?d - drone)
    :precondition (and (flying ?d) (< (health ?d) (minimum-health ?d)))
    :effect (and (health-failure ?d) (not (flying ?d)) (not (in-formation ?d))))
```

Because every collective action requires `in-formation` for every grid member, and the goal demands it for every drone, one faulting drone immediately prevents the grid from establishing, closing up, or completing. The formation is modelled as genuinely rigid: it does not fly on with a hole in it.

ACTION 12 of 14 - detect-neighbor-link-loss

```
(:action detect-neighbor-link-loss
    :parameters (?d - drone)
    :precondition (and (flying ?d)
                       (not (neighbor-comm-ok ?d)))
    :effect (and (communication-lost ?d)
                 (not (flying ?d))
                 (not (in-formation ?d))))
```

The fourth detector, and the one the V-Formation domain later copied. It fires when an airborne drone has lost contact with its **neighbours in the grid** - as opposed to `detect-communication-loss`, which concerns the link to the **ground controller**.

The two are modelled as equally serious: both set `communication-lost`, both ground the drone, both eject it from the formation. In a grid with 10-metre spacing that is the right call - a drone that cannot hear the aircraft 10 m off its wingtip is a hazard whether or not the ground station can still hear it.

ACTION 14 of 14 - notify-ground-controller

```
(:action notify-ground-controller
    :parameters (?d - drone ?c - controller)
    :precondition (and (or (gps-lost ?d) (communication-lost ?d) (health-failure ?d)))
    :effect (controller-notified ?d))
```

The single upward message. All four detectors funnel into it, since both communication detectors set `communication-lost`. As in the V there are no downward controller orders in this domain: a fault is reported and the formation is broken, but nothing responds to it.

3. problem.pddl explained

The grid template describes the smallest possible case - a 1x2 grid, the leader plus one drone at its side. `expand_grid_drones` rebuilds it into whatever shape the drone count calls for.

problem.pddl - the objects

```
(define (problem grid-formation-mission-001)
    (:domain grid-formation-drone-mission)

    (:objects
        drone-lead   - drone
        drone-member - drone
        ground-controller - controller
        source      - location
        destination - location
    ))
```

Just two drones, and no compass-direction objects - the grid has no `slot-direction` bookkeeping, so it needs none.

`drone-member` is the **reference member** that `expand_grid_drones` clones. Unusually, the name is never renamed: whatever cell happens to come first in row-major order for a given grid size keeps the name `drone-member`, and the rest are named `drone-r{row}c{col}`. That is visible in the worked run, where `drone-member` appears alongside `drone-r1c2`, `drone-r0c3` and so on.

Only two locations, as in the V. A detour's via-points are generated at run time if one is needed.

problem.pddl - starting state and slot assignment

```
;; Both drones start at the source
(at drone-lead source)  (at drone-member source)
(landed drone-lead)     (landed drone-member)
(= (altitude drone-lead) 0)  (= (altitude drone-member) 0)

;; Direct route between source and destination
(connected source destination)  (connected destination source)
(safe-route source destination) (safe-route destination source)

;; Assign formation leader and grid member
(formation-leader drone-lead)
(grid-member drone-member)
(has-slot drone-lead)  (has-slot drone-member)

;; Grid slot positions
(= (slot-along-offset drone-lead)     0)
(= (slot-cross-offset drone-lead)     0)
(= (slot-along-offset drone-member)   0)
(= (slot-cross-offset drone-member)  10)
```

Both drones begin landed at the source at zero altitude, and the direct route is legal in both directions - there is no built-in detour shape.

`drone-lead` carries `formation-leader` and notably does **not** carry `grid-member`, which is what excludes it from every `forall` over members.

The template's offsets describe a 1x2 grid: the member sits in the same row (along offset 0) and 10 m to the right (cross offset 10). `expand_grid_drones` overwrites both for every cell on every run.

> **A formatting detail that once broke this plan**
>
> Every per-drone numeric fact in this file is written on **one line**, including the two `hover-altitude` facts below. That was not always so: until 2026-09-16 they were written across three lines each, which broke the line-based cloner and made every grid larger than 1x2 report as unsolvable. Chapter 6 tells the full story. When editing this template, keep every `(= (fluent object) value)` fact on a single line.

problem.pddl - the neighbour pair and the launch clock

```
;; Neighbor communication
(grid-neighbor drone-lead drone-member)
(grid-neighbor drone-member drone-lead)

(neighbor-comm-ok drone-lead)
(neighbor-comm-ok drone-member)

;; Staggered launch settings
(= (seconds-since-leader-airborne) 0)
(= (grid-launch-delay) 5)
```

For the 1x2 template the neighbour relation is a single symmetric pair. For anything larger, `expand_grid_drones` **strips these facts entirely** and rebuilds the complete adjacency mesh - see section 4.4 for why cloning them would be wrong.

The clock starts at 0 against a delay of 5, forcing exactly five `hold-position` actions into every plan.

problem.pddl - health, battery and the flight profile

```
;; System health starts normal
(gps-ok drone-lead) (communication-ok drone-lead) (drone-healthy drone-lead)
(gps-ok drone-member) (communication-ok drone-member) (drone-healthy drone-member)

(= (health drone-lead) 100)   (= (minimum-health drone-lead) 40)
(= (battery drone-lead) 100)  (= (max-battery drone-lead) 100)
(= (minimum-return-battery drone-lead) 30)

;; Altitude settings
(= (hover-altitude drone-lead)    2)
(= (hover-altitude drone-member)  2)
(= (target-altitude drone-lead)    50)
(= (target-altitude drone-member)  50)

;; Battery used during climb
(= (climb-energy drone-lead) 4)   (= (climb-energy drone-member) 4)

;; Battery used for each cruise leg
(= (cruise-leg-energy drone-lead)    12)
(= (cruise-leg-energy drone-member)  14)
```

The same profile as the V: 2 m hover, 50 m cruise, 4 units to climb, full health and battery.

The `cruise-leg-energy` asymmetry is inherited too - 12 for the leader, 14 for each member, reflecting that a trailing aircraft covers more ground closing up and gets none of the leader's aerodynamic benefit. Since every member is cloned from `drone-member`, every non-leader drone in the grid is billed 14.

The two `hover-altitude` lines are the facts the 2026-09-16 defect corrupted.

problem.pddl - the goal

```
(:goal (and
    (grid-formation-established)
    (in-formation drone-lead) (in-formation drone-member)

    (at-cruise-altitude drone-lead) (at-cruise-altitude drone-member)
    (>= (altitude drone-lead) 50) (>= (altitude drone-member) 50)

    (formation-at destination)
    (at drone-lead destination) (at drone-member destination)

    (formation-mission-complete)
    (mission-completed drone-lead) (mission-completed drone-member)))
```

As rich as the V's: not merely arrival, but that the grid was **established**, that every drone is **still in formation** on arrival, and that every drone is at cruise altitude both by flag and by number.

Requiring `in-formation` for every drone in the goal is what makes a detected fault fatal - the detectors clear that flag and nothing can restore it.

`expand_grid_drones` clones the per-drone conjuncts for every new cell, so a 12-drone grid's goal carries the same five conjuncts for each of its twelve drones.

4. Every Python function in the Grid-Formation path

4.1 The constant, and turning a count into a shape

GRID_SPACING_M and _GRID_MEMBER_TOKEN

```
GRID_SPACING_M = 10.0
_GRID_MEMBER_TOKEN = "drone-member"
```

The spacing between adjacent drones, front-to-back and side-to-side alike. The comment above it makes a point worth noting: unlike the V's `FORMATION_BACK_M`/`FORMATION_SIDE_M`, which duplicate numbers that also live in the problem file, this single constant drives **both** representations - `expand_grid_drones` uses it to write the descriptive slot offsets, and `plan_service` uses it to derive each member's actual flown track. There is nothing to keep in sync by hand.

grid_dimensions(drone_count)

```
def grid_dimensions(drone_count: int) -> tuple[int, int]:
    if drone_count < 1:
        raise ValueError(f"grid drone_count must be >= 1, got {drone_count}")
    rows = int(math.isqrt(drone_count))
    while rows > 1 and drone_count % rows != 0:
        rows -= 1
    return rows, drone_count // rows
```

Four lines that turn a drone count into a grid shape. It starts at the integer square root and walks **down** until it finds a divisor, then pairs that with the matching quotient.

Why this gives the most-square exact tiling. Any factor pair of the count has one member at or below the square root and one at or above it. The largest divisor at or below the square root is therefore the one closest to it, and its partner is closest from the other side - so that pair is the most nearly square rectangle that tiles the count exactly. A perfect square falls out naturally, since the square root is itself a divisor and the loop never runs.

The prime case degenerates. For 7 or 11 the loop walks all the way down to 1, giving a 1-by-N single line. That is a genuine limitation rather than a bug - there is no rectangle with exactly 7 cells other than a line - and it is one reason `MIN_GRID_DRONES` exists.

`rows <= cols` always holds, and `expand_grid_drones` decides which physical axis is which: row 0 is the front row, nearest the direction of travel.

grid_cell_names(rows, cols)

```
def grid_cell_names(rows: int, cols: int) -> dict[tuple[int, int], str]:
    leader_cell = (0, (cols - 1) // 2)
    names = {leader_cell: "drone-lead"}
    member_cells = [(row, col) for row in range(rows) for col in range(cols)
                    if (row, col) != leader_cell]
    for i, cell in enumerate(member_cells):
        names[cell] = _GRID_MEMBER_TOKEN if i == 0 else f"drone-r{cell[0]}c{cell[1]}"
    return names
```

Maps every (row, column) cell to the drone object name that will occupy it. It is exposed publicly so that `plan_service` can derive every drone's name and cell without re-parsing the expanded problem text.

Where the leader sits. Front row, in the column nearest the middle - `(cols - 1) // 2`. That is exact for an odd column count and off by at most half a slot for an even one, which is the same unavoidable asymmetry a V has when its wings cannot be split perfectly evenly.

The naming rule is slightly unusual. The first member cell in row-major order keeps the template's own name `drone-member`; every other cell gets `drone-r{row}c{col}`. Keeping the template's name for one cell is what lets `_clone_named_lines` work - it clones the lines mentioning `drone-member`, so that name must survive as a real drone.

4.2 Growing the grid

expand_grid_drones - step 1: validation and objects

```
def expand_grid_drones(problem_text: str, drone_count: int) -> str:
    if drone_count < 2:
        raise ValueError(f"grid-formation drone_count must be >= 2, got {drone_count}")
    rows, cols = grid_dimensions(drone_count)
    leader_col = (cols - 1) // 2
    cell_name = grid_cell_names(rows, cols)
    member_cells = [cell for cell, name in cell_name.items() if name != "drone-lead"]
    member_names = [cell_name[cell] for cell in member_cells]
    first_name, rest_names = member_names[0], member_names[1:]

    # 1. Objects: widen "drone-member - drone" to every member name
    obj_open, obj_close = _section_span(problem_text, "objects")
    widened, n_matched = re.subn(
        rf"\b{re.escape(_GRID_MEMBER_TOKEN)}\b(\s*-\s*drone\b)",
        lambda m: " ".join([first_name, *rest_names]) + m.group(1),
        problem_text[obj_open:obj_close], count=1)
    if not n_matched:
        raise ValueError("grid-formation template has no 'drone-member - drone' object")
    result = problem_text[:obj_open] + widened + problem_text[obj_close:]
```

The docstring makes an important architectural point. Unlike the V - which only ever grows two fixed wings further out along the same two lines - a grid's shape changes **non-monotonically** between counts. Nine drones is a 3x3 square, not '2x2 plus one more somewhere'. So this function always rebuilds the full member list, offsets and neighbour links from the drone count directly, the same way for every count, rather than incrementally extending whatever the template already had.

The minimum is 2 here - a grid needs at least one neighbour pair - though `MainWindow` additionally enforces `MIN_GRID_DRONES = 4` before it will attempt a run, so a grid always reads as two-dimensional rather than a bare line.

Step 1 widens `drone-member - drone` to list every member name. `re.subn` is used rather than `re.sub` so the match count can be checked: a template without the expected declaration fails loudly here instead of producing a file that references undeclared objects.

expand_grid_drones - steps 2 and 3: clone, then place each cell

```
    # 2. Clone every :init fact and goal conjunct mentioning drone-member
    result = _clone_named_lines(result, _section_span(result, "init"),
                                _GRID_MEMBER_TOKEN, rest_names)
    result = _clone_named_lines(result, _goal_and_span(result),
                                _GRID_MEMBER_TOKEN, rest_names)

    # 3. Fix up every drone's along/cross offset for its actual cell
    result = _set_unary_fact_value(result, "slot-along-offset", "drone-lead", 0.0)
    result = _set_unary_fact_value(result, "slot-cross-offset", "drone-lead", 0.0)
    for cell, name in zip(member_cells, member_names):
        row, col = cell
        result = _set_unary_fact_value(result, "slot-along-offset", name,
                                       -row * GRID_SPACING_M)
        result = _set_unary_fact_value(result, "slot-cross-offset", name,
                                       (col - leader_col) * GRID_SPACING_M)
```

Step 2 clones every initial-state fact and goal conjunct mentioning `drone-member`, once per further member - health, battery, altitudes, `grid-member`, `has-slot`, `neighbor-comm-ok`, and the goal's five per-drone conjuncts. The goal cloning happens inside the goal's `(and ...)`, using the same `_goal_and_span` helper the other expanders use, so the result stays a single well-formed goal expression.

Step 2 is exactly where the 2026-09-16 defect lived. `_clone_named_lines` works line-by-line, so a fact spread over three lines had only its middle line cloned. See chapter 6.

Step 3 then corrects the only facts that differ between cells. The along offset is `-row * 10` - negative meaning behind, matching the V wings' trailing-negative convention, with the leader's own row always 0. The cross offset is `(col - leader_col) * 10`, negative to the left of the leader's column and positive to its right.

expand_grid_drones - step 4: rebuilding the adjacency mesh

```
    # 4. Neighbor links: front/back and left/right adjacency only (no
    #    diagonals), one symmetric pair per adjacent cell pair.
    result = _strip_pair_facts(result, "grid-neighbor")
    neighbor_facts = []
    for row in range(rows):
        for col in range(cols):
            here = cell_name[(row, col)]
            for neighbor_row, neighbor_col in ((row + 1, col), (row, col + 1)):
                if neighbor_row < rows and neighbor_col < cols:
                    there = cell_name[(neighbor_row, neighbor_col)]
                    neighbor_facts.append(f"\n        (grid-neighbor {here} {there})")
                    neighbor_facts.append(f"\n        (grid-neighbor {there} {here})")
    result = _add_init_facts(result, "".join(neighbor_facts))
    return result
```

The step that exists to **undo** something step 2 got wrong, exactly as in the V.

The bug it prevents. Step 2 cloned every line mentioning `drone-member`, including the template's `(grid-neighbor drone-lead drone-member)`. Cloned for a cell three rows back, that becomes `(grid-neighbor drone-lead drone-r2c3)` - wiring a distant corner directly to the leader. Every cell would end up a neighbour of the leader and of nobody else, which misdescribes the grid completely.

The fix. Strip every `grid-neighbor` fact and compute the complete adjacency mesh fresh.

Why only two directions are checked. The inner loop considers only the cell below and the cell to the right. Since each pair is written symmetrically in both directions, scanning down-and-right over every cell covers every edge exactly once - the up and left edges are produced when the **other** cell of each pair is visited. Scanning all four directions would duplicate every fact.

No diagonals. A cell's neighbours are strictly front, back, left and right. A corner cell has 2 neighbours, an edge cell 3, an interior cell 4 - never more, however large the grid.

_strip_pair_facts(problem_text, predicate)

```
def _strip_pair_facts(problem_text: str, predicate: str) -> str:
    return re.sub(rf"[ \t]*\({re.escape(predicate)}\s+\S+\s+\S+\)\n?", "", problem_text)
```

Deletes every pairwise fact of a given predicate, along with its line. Shared by both formation expanders - `grid-neighbor` here, `wing-neighbor` in the V - to clear the template's hardcoded smallest-case pairs before the complete set is computed fresh.

4.3 Retargeting the grid

retarget_grid_problem(...) [the main entry point]

```
def retarget_grid_problem(problem_text, new_source, new_destination, *,
                          no_fly_zone=None, no_fly_buffer_m=NO_FLY_BUFFER_M,
                          source_name="source", destination_name="destination") -> str:
    locations = read_locations(problem_text)
    if source_name not in locations or destination_name not in locations:
        raise ValueError(...)
    transform = _similarity_transform(locations[source_name], locations[destination_name],
                                      new_source, new_destination)
    moved = {name: transform(point) for name, point in locations.items()}
    moved[source_name] = new_source
    moved[destination_name] = new_destination
    result = problem_text
    for name, point in moved.items():
        result = _set_latlon(result, name, point)

    for a, b in _read_edges(problem_text):
        if a not in moved or b not in moved: continue
        old_distance = _fact_value(problem_text, "distance", a, b)
        old_energy = _fact_value(problem_text, "energy-required", a, b)
        rate = (old_energy / old_distance
                if old_distance and old_energy is not None else DEFAULT_ENERGY_RATE)
        new_distance = haversine_m(moved[a], moved[b])
        if old_distance is not None:
            result = _set_fact_value(result, "distance", a, b, new_distance)
        if old_energy is not None:
            result = _set_fact_value(result, "energy-required", a, b, new_distance * rate)

    if no_fly_zone and len(no_fly_zone) >= 3:
        seg_a = _to_local(new_source, new_source)
        seg_b = _to_local(new_destination, new_source)
        polygon_local = _convex_hull([_to_local(p, new_source) for p in no_fly_zone])
        endpoint_inside = (_point_in_polygon(seg_a, polygon_local)
                           or _point_in_polygon(seg_b, polygon_local))
        if len(polygon_local) >= 3 and not endpoint_inside:
            routing_polygon = _offset_polygon(polygon_local, no_fly_buffer_m)
            if (_point_in_polygon(seg_a, routing_polygon)
                    or _point_in_polygon(seg_b, routing_polygon)):
                routing_polygon = polygon_local
            if _segment_crosses_polygon(seg_a, seg_b, routing_polygon):
                detour_local = _shortest_detour(seg_a, seg_b, routing_polygon)
                via_points = [_from_local(p, new_source) for p in detour_local[1:-1]]
                if via_points:
                    result = _set_detour_route(result, source_name, destination_name,
                                               new_source, new_destination, via_points)
    return result
```

The **simplest** of the three retargeting functions, and deliberately so.

Phase 1 - move. Every location is carried through the rotate-plus-scale-plus-translate that maps the template's source-to-destination onto the picked one, then the two endpoints are pinned exactly to the operator's clicks rather than trusting a floating-point round trip.

Phase 2 - re-cost. Each edge's original energy-per-metre rate is recovered by dividing authored energy by authored distance, and the new energy is the new distance at that same rate - preserving any authored asymmetry between legs.

Phase 3 - detour. Identical to the V's: hull the marked polygon, refuse to detour if either endpoint is inside it, buffer the boundary (falling back to the true boundary if the buffer would swallow an endpoint), and if the direct line still crosses it, build a visibility-graph shortest path around it and splice in the via-points.

What is absent is the point. There is no `_set_direct_route_only` branch, because the template's only declared route **is** the direct one - there is no built-in detour to prune. And there is no slot-direction recomputation, because a grid member's position is read off its numeric offsets directly rather than through a compass label. Once the coordinates and the route are relocated, there is nothing else to do.

The buffer is a parameter, not a constant, because only the leader's corridor is planned against the zone while the grid's outer columns ride out to the side of it.

4.4 Deriving each member's track

formation_member_route(lead_route, along_m, cross_m) [the geometry that makes the grid]

```
def formation_member_route(lead_route, *, along_m, cross_m) -> list[LatLon]:
    if len(lead_route) < 2: return list(lead_route)
    n = len(lead_route)
    route = []
    for i, here in enumerate(lead_route):
        incoming = _unit(_to_local(here, lead_route[i-1])) if i > 0 else None
        outgoing = _unit(_to_local(lead_route[i+1], here)) if i + 1 < n else None

        fwd = None
        if incoming and outgoing:
            fwd = _unit((incoming[0] + outgoing[0], incoming[1] + outgoing[1]))
        if fwd is None:
            fwd = incoming or outgoing
        if fwd is None:
            route.append(here); continue

        port = (-fwd[1], fwd[0])                      # 90 deg left of forward
        back = (-fwd[0] * along_m, -fwd[1] * along_m)
        route.append(_from_local(
            (back[0] + port[0] * cross_m, back[1] + port[1] * cross_m), here))
    return route
```

Produces one member's track, held parallel to the leader's. For each of the leader's waypoints it emits a corresponding point `along_m` behind and `cross_m` to the right, so the member-to-leader spacing is the fixed formation distance **everywhere** along the route - at the source and destination alike, not only once airborne.

This is the single-offset function the V's `formation_wing_routes` is built on top of. The docstring explains why it was factored out: a V's two wings are always a symmetric plus-or-minus pair around the lead track, but a grid's members generally are not - a 3x4 grid's twelve members each sit at their own distinct row-and-column offset, so each needs its own independent call.

The turn bisector is the subtle part. At an interior waypoint - a detour bend - 'forward' is the sum of the incoming and outgoing unit directions, which bisects the turn. The tempting shortcut of using the straight line from the point **before** the bend to the point **after** it is wrong in a way that is easy to miss: for a single bend it collapses straight back to the original source-to-destination bearing no matter how far the via-point swings out. A member would then hold its pre-detour heading exactly at the bend - pinching in towards the lead track or crossing it - instead of turning with the formation.

At an endpoint there is only one leg, so that leg's own direction is used. `port` is the unit vector 90 degrees left of forward, from the standard rotation `(x, y) -> (-y, x)`.

The returned route always has the same number of points as the leader's, which is what lets every drone's track be flown as a parallel sequence.

extract_formation_corridor(steps) [engine/pddl_planner.py]

```
def extract_formation_corridor(steps: list[PlanStep]) -> list[str]:
    legs = [step.args[1:3]
            for step in sorted(steps, key=lambda s: s.index)
            if step.action == "formation-cruise" and len(step.args) >= 3]
    if not legs: return []
    corridor = [legs[0][0]]
    corridor.extend(to for _from, to in legs)
    return corridor
```

Pulls the leader's route out of the plan. It takes no drone argument because there is only ever **one** corridor in a formation plan - only the leader moves in a `formation-cruise` action. Shared unchanged with the V-Formation plan, since both domains use the same action name. The chaining is the usual one: the first leg's origin followed by every leg's destination.

4.5 Orchestration

MIN_GRID_DRONES and _grid_drone_names(drone_count)

```
MIN_GRID_DRONES = 4   # smallest shape that reads as a grid rather than a line

def _grid_drone_names(drone_count: int) -> list[str]:
    rows, cols = grid_dimensions(drone_count)
    cell_name = grid_cell_names(rows, cols)
    leader_cell = next(cell for cell, name in cell_name.items() if name == "drone-lead")

    def manhattan(cell: tuple[int, int]) -> int:
        return abs(cell[0] - leader_cell[0]) + abs(cell[1] - leader_cell[1])

    ordered_cells = sorted(cell_name, key=manhattan)
    return [cell_name[cell] for cell in ordered_cells]
```

Produces every drone name for a grid, leader first and then every other cell ordered nearest-to-farthest from it by **Manhattan distance** in grid steps - the number of front/back plus left/right hops between cells, which is the natural metric here because that is exactly how the neighbour mesh is wired.

The rationale is the same usability one behind `_formation_drone_names`'s rank interleaving: the first drones the operator ticks should land nearest the leader, whatever the size of the grid, rather than being scattered to distant corners.

`MIN_GRID_DRONES = 4` is enforced by the GUI before a run is attempted. `expand_grid_drones` itself only requires 2, but a 1x2 or 1x3 arrangement is a line rather than a grid, and 4 is the smallest count that forms a genuine 2x2 square.

_Worker._run_grid_formation - part 1: expand, retarget, solve

```
def _run_grid_formation(self, plan_name, source, destination,
                        no_fly_zone=None, drone_count=MIN_GRID_DRONES) -> PlanRunResult:
    rows, cols = grid_dimensions(max(MIN_GRID_DRONES, drone_count))
    leader_col = (cols - 1) // 2
    domain_path, template_path = self._plan_files(plan_name)
    template_text = template_path.read_text(encoding="utf-8")
    template_text = expand_grid_drones(template_text, max(MIN_GRID_DRONES, drone_count))
    concrete_text = retarget_grid_problem(
        template_text,
        PddlLatLon(lat=source.lat, lon=source.lon),
        PddlLatLon(lat=destination.lat, lon=destination.lon),
        no_fly_zone=(...),
        no_fly_buffer_m=NO_FLY_BUFFER_M
            + GRID_SPACING_M * max(leader_col, cols - 1 - leader_col))

    run_dir = self._start_run(plan_name, concrete_text)
    steps, stdout = run_enhsp(
        domain_path, run_dir / "problem.pddl",
        timeout_s=max(30.0, 1.0 * rows * cols),
        extra_args=["-gro", "naive"])
    (run_dir / "plan.txt").write_text(stdout, encoding="utf-8")
```

Expand before retarget, as in the V and Search.

The padded no-fly buffer. ENHSP plans only the leader's centreline, but the grid's outermost columns ride out to either side of it. The padding is `GRID_SPACING_M * max(leader_col, cols - 1 - leader_col)` - the distance from the leader's column to whichever edge column is farther, which is the grid's true lateral reach. Buffering by the base 10 m alone would clear the leader while flying the outer columns through the marked area.

The `-gro naive` flag is the most consequential line in this function, and the comment explains it at length. ENHSP's default internal grounder was found to blow up - a multi-minute, multi-gigabyte `OutOfMemoryError` inside its own combination-building, well before search even starts - on a formation-style domain once the drone count climbs past a dozen or so, regardless of how much JVM heap it is given. The V's domain is affected equally. Its 'naive' grounder does not share that blowup: a 36-drone 6x6 grid grounds and solves in about two seconds with the JVM's ordinary default heap. The flag was chosen rather than trying to out-resource the default grounder's own growth.

The timeout scales with the grid area - one second per cell, floor of 30 - so a large grid is given proportionally more budget.

_Worker._run_grid_formation - part 2: deriving every member's track

```
    corridor = extract_formation_corridor(steps)
    if not corridor:
        raise PlanNotFound("ENHSP found a plan but it contains no 'formation-cruise' legs.")
    locations = read_locations(concrete_text)
    lead_route = [PddlLatLon(lat=locations[n].lat, lon=locations[n].lon) for n in corridor]

    cell_name = grid_cell_names(rows, cols)
    leader_cell = next(cell for cell, name in cell_name.items() if name == "drone-lead")
    routes = {"drone-lead": _as_gui(lead_route)}
    for (row, col), name in cell_name.items():
        if (row, col) == leader_cell: continue
        # `along_m` is "how far behind" (positive = behind), the OPPOSITE
        # sign from the PDDL problem's own slot-along-offset fact
        # (negative = behind) - that fact is descriptive bookkeeping only.
        member_route = formation_member_route(
            lead_route,
            along_m=row * GRID_SPACING_M,
            cross_m=(col - leader_cell[1]) * GRID_SPACING_M)
        routes[name] = _as_gui(member_route)
    per_drone_waypoints = {name: routes[name] for name in _grid_drone_names(rows * cols)}

    return PlanRunResult(plan_name=plan_name, steps=steps,
                         waypoints=_as_gui(lead_route), location_names=corridor,
                         run_dir=run_dir, per_drone_waypoints=per_drone_waypoints)
```

One call per cell. Unlike the V, where `formation_wing_routes` produces a symmetric pair per rank, each grid cell needs its own `formation_member_route` call with its own distinct row-and-column offset.

The sign convention differs between the two representations, and the comment flags it explicitly. `formation_member_route`'s `along_m` is 'how far behind', so behind is **positive** - the same sign the V's `FORMATION_BACK_M` uses. The PDDL problem's own `slot-along-offset` fact uses the opposite convention, negative for behind. The two never meet: the PDDL fact is descriptive bookkeeping that no action reads, and it is not fed into this call. Passing `row * GRID_SPACING_M` unnegated is correct precisely because of that sign difference.

The cross offset is relative to the leader's column, so columns left of the leader get a negative value and columns right of it a positive one.

The final dictionary comprehension reorders the routes into `_grid_drone_names` order - nearest the leader first - so that the GUI's checked drones are assigned to cells sensibly.

The result carries both forms. `waypoints` holds the leader's route for display code that expects a single path; `per_drone_waypoints` holds all of them, the same shape Search and the V produce, so the swarm flies concurrently through the same per-drone execution path.

5. How the drones communicate

The grid has the richest neighbour model of the four plans - it is the plan the V's model was copied from - so it is worth separating the two channels carefully.

5.1 Two distinct channels

Fact	What it means	Checked by
`(communication-ok ?d)`	This drone's radio link to the GROUND CONTROLLER is healthy.	pre-flight-check, takeoff, climb-to-cruise-altitude, hold-position, take-grid-slot, formation-cruise, close-up-grid
`(neighbor-comm-ok ?d)`	This drone's links to its UP-TO-4 NEIGHBOURS IN THE GRID are healthy.	establish-grid-formation, formation-cruise, close-up-grid
`(grid-neighbor ?d1 ?d2)`	?d1 and ?d2 are adjacent front/back or left/right. Descriptive only.	nothing

The division of labour is clear from the third column. The ground link is required for every **individual** action, including getting off the ground. The neighbour link is required only for the three **collective** actions - forming up, cruising, and closing up - because those are precisely the moments at which drones must coordinate with one another.

5.2 The neighbour topology

The grid-neighbor mesh

```
A 2x4 grid of 8 drones.  Each line is a symmetric pair of
(grid-neighbor a b) / (grid-neighbor b a) facts:

   drone-r0c0 --- drone-lead --- drone-r0c2 --- drone-r0c3     row 0
        |              |              |              |
        |              |              |              |
   drone-r1c0 --- drone-r1c1 --- drone-r1c2 --- drone-r1c3     row 1

Built by expand_grid_drones step 4 - for each cell, look DOWN and RIGHT
only; writing each pair symmetrically covers every edge exactly once.

Neighbour counts, whatever the grid size:
    corner cell   2 neighbours
    edge cell     3 neighbours
    interior cell 4 neighbours  (front, back, left, right)

NO DIAGONALS.  drone-lead and drone-r1c2 are not neighbours.
```

This is the property the plan is designed around, and the one worth emphasising in a write-up: **each drone coordinates with at most four others, no matter how large the swarm is**. A 36-drone 6x6 grid still has every drone talking to at most four neighbours. The coordination burden per aircraft is constant; only the number of aircraft grows.

The topology is purely descriptive - no action reads `grid-neighbor`. What the actions check is the per-drone `neighbor-comm-ok` flag, and the domain's own comment gives the reason: a per-drone check keeps each action's cost linear in the drone count, whereas verifying every declared pair would be quadratic. That matters more here than in a V, because a rows-by-cols grid has roughly twice as many adjacencies as it has drones.

5.3 What happens when a link fails

Failure propagation

```
GROUND LINK LOST                      NEIGHBOUR LINK LOST
(communication-ok false)              (neighbor-comm-ok false)
        |                                      |
        v                                      v
detect-communication-loss            detect-neighbor-link-loss
        |                                      |
        +--------------+-----------------------+
                       |
                       v
            (communication-lost ?d)
            (not (flying ?d))
            (not (in-formation ?d))       <-- ejected from the formation
                       |
      +----------------+----------------------------+
      |                                             |
      v                                             v
notify-ground-controller              every collective action is now BLOCKED:
(controller-notified ?d)                establish-grid-formation - forall fails
                                        close-up-grid            - forall fails
                                        complete-grid-mission    - forall fails
                                      and the GOAL itself requires
                                        (in-formation ?d) for every drone
```

Both failures are treated identically and both are **fatal to the mission**. Because every collective action's `forall` requires `in-formation` for every grid member, and the goal demands it for every drone, one lost link anywhere in the grid makes the mission unachievable. Nothing can restore `in-formation` once the drone has stopped flying.

That is the correct model for a rigid formation - the grid does not fly on with a hole in it. It does mean the domain has no **degraded** mode: there is no action to reconfigure the grid around a lost member, and no downward controller order to respond with. The model detects and reports; it does not recover.

5.4 At runtime: still a star through the GCS

The runtime transport

```
                  GROUND CONTROL STATION
                    (engine/gcs.py)
                one UDP socket, port 14690
                          |
      +-------------------+-------------------+
      |                   |                   |
  udp:14701           udp:14702           udp:1470N
  drone-lead          drone-member        drone-r1c2 ...

GCS  ------ COMMAND (JSON/UDP) ------>  drone
GCS  <-- TELEMETRY / HEARTBEAT / ACK --  drone
drone  <--------- X ---------->  drone      (no such link exists)

protocol.py:  drone_port(sysid) = 14700 + sysid
              GCS_PORT = 14690
drone_link.py: HEARTBEAT_INTERVAL_S = 1.0, TELEMETRY_INTERVAL_S = 0.5
gcs.py:        HEARTBEAT_TIMEOUT_S  = 3.0
```

This is where the model and the implementation diverge, and it should be stated plainly in any write-up. The `grid-neighbor` mesh and the `neighbor-comm-ok` flag describe a **peer-to-peer** structure. The runtime has no peer-to-peer transport at all: searching the codebase for peer links finds 'neighbor' only in the PDDL-generating modules, never in the networking ones.

Every drone binds its own UDP port at `14700 + sysid` and speaks only to the ground station, which holds one socket for the entire swarm, keeps the latest telemetry and heartbeat per drone, and fans commands back out. Every message carries `drone_id`, the drone's MAVLink SYSID, so a single socket can address and demultiplex the whole fleet. The wire format is one JSON datagram per message, chosen so that the later substitution of MAVLink is a translation exercise rather than a rewrite.

Formation cohesion at runtime is achieved not by drones talking to each other but by each flying its own precomputed parallel track - the tracks `formation_member_route` generated, which are parallel by construction and therefore hold station without any negotiation. Link loss is observable rather than assumed: the GCS tracks each drone's `last_heartbeat_at` against a three-second timeout, with heartbeats every second and telemetry twice a second.

So the accurate summary is: **the neighbour mesh is a planning-time abstraction for what a real formation-flying implementation would need.** It lets the planner refuse to launch a formation whose members could not maintain contact, and it documents the topology such a system would use - four links per aircraft regardless of swarm size. Implementing it would mean adding a peer transport alongside the existing star.

6. A worked run

A real 8-drone Grid-Formation run from `plans/gridformation/runs/` - a 2x4 grid:

ENHSP output - 8 drones, 2x4 grid, direct route

```
Problem Solved
Found Plan:
 0.0: (pre-flight-check drone-lead source)
 1.0: (takeoff drone-lead source)
 2.0: (climb-to-cruise-altitude drone-lead)
 3.0: (hold-position drone-lead)          <-- the leader burns the launch clock
 4.0: (hold-position drone-lead)
 5.0: (hold-position drone-lead)
 6.0: (hold-position drone-lead)
 7.0: (hold-position drone-lead)          <-- 5 x 1 s = grid-launch-delay
 8.0: (pre-flight-check drone-r1c2 source)
 9.0: (takeoff drone-r1c2 source)
10.0: (climb-to-cruise-altitude drone-r1c2)
11.0: (pre-flight-check drone-member source)
12.0: (takeoff drone-member source)
13.0: (climb-to-cruise-altitude drone-member)
...                                  (the other members, 3 steps each)
30.0: (take-grid-slot drone-member)
31.0: (take-grid-slot drone-r1c3)
32.0: (take-grid-slot drone-r1c1)
33.0: (take-grid-slot drone-r0c2)
34.0: (take-grid-slot drone-r1c2)
35.0: (take-grid-slot drone-r0c3)
36.0: (take-grid-slot drone-r1c0)
37.0: (establish-grid-formation drone-lead source)
38.0: (formation-cruise drone-lead source destination)
39.0: (close-up-grid drone-lead destination)
40.0: (complete-grid-mission drone-lead destination)
Plan-Length:41
```

Five observations worth drawing out.

The staggered launch is unmistakable. Steps 0 to 2 get the leader airborne and climbing; steps 3 to 7 are five `hold-position` actions advancing the clock from 0 to 5; only at step 8 does the first member begin its pre-flight check. The disjunctive precondition in `takeoff` produced exactly the discipline it was written for.

Each member costs exactly three steps - pre-flight, takeoff, climb - repeated seven times for the seven members, filling steps 8 to 29.

The cell naming is visible. `drone-member` appears alongside `drone-r1c2`, `drone-r0c3` and the rest - the template's own member name survives as the first cell in row-major order, exactly as `grid_cell_names` specifies.

The slot-taking order is arbitrary. The drones are independent at this stage and the planner has no reason to prefer an order; it does not matter, because `establish-grid-formation` requires all of them.

The entire cruise is three actions. Steps 38, 39 and 40 - cruise, close up, complete - are the whole journey, because the route is a single direct leg. A run with a marked restriction in the way would show a repeating cruise/close-up pair for each detour leg, which is the `grid-closed` interlock at work.

Only `drone-lead` appears in the last four steps. This is the defining property of the plan: the leader is the only drone the planner ever moves. All seven member tracks are produced afterwards by `formation_member_route`, from the single corridor `extract_formation_corridor` reads out of step 38.

6.1 Record to date

Metric	Value
Runs logged	19
Solved	15
Unsolvable / no plan	4 (all pre-fix, 2026-09-16 10:06-10:07)

Metric	Value
Open code issues	None

All four failures fall within two minutes of each other on 2026-09-16, immediately before a fix. Every run from 10:12 onward the same day solved cleanly, up to at least an 8-drone 2x4 grid with a plan length of 41 - the run reproduced above.

6.2 The 2026-09-16 defect, in full

This is the most instructive bug in the project, because its cause was a formatting choice rather than a logic error. It is worth understanding in detail.

The symptom

Any grid larger than the template's own 1x2 came back from ENHSP as 'Unsolvable Problem' - flagged during AIBR preprocessing, before search even began. A 4-drone 2x2 grid failed as surely as a 16-drone one.

The cause

The change that fixed it

```
;; BEFORE - the two hover-altitude facts, written across three lines each
(=
    (hover-altitude drone-member)
    2
)

;; AFTER - one line, matching every other per-drone fluent in the file
(= (hover-altitude drone-member)  2)
```

`expand_grid_drones` clones facts **line by line**, through `_clone_named_lines`. That function scans each line inside a span for the token `drone-member` and duplicates the whole line with the token substituted.

Of the three lines above, only the middle one mentions `drone-member`. So for every extra grid member the cloner produced exactly one line: `(hover-altitude drone-r0c2)` - a malformed, valueless fact with no assigned number, and no surrounding `(=` or closing parenthesis.

The result was that `hover-altitude` was left **undefined** for every drone except the template's original two. `takeoff` assigns `altitude` from `hover-altitude`, and `climb-to-cruise-altitude` compares `altitude` against `target-altitude` - so an undefined `hover-altitude` propagated into an undefined `altitude`, which ENHSP's AIBR preprocessing correctly flagged as unsolvable.

Every **other** per-drone fluent in the file - `slot-along-offset`, `climb-energy`, `battery`, and the rest - was already written on one line, and `plans/vformation/problem.pddl` used the one-line convention throughout. Only these two facts were formatted differently, which is exactly why the bug was invisible on inspection.

The verification

The fix was checked directly against ENHSP rather than only through the application: the old template reproduced 'Unsolvable Problem' even at four drones, and the corrected template returned a valid plan for the same scenario. The in-app runs from 10:12 onward confirmed it end to end.

The lesson

When editing any of the four problem templates

Keep every `(= (fluent object) value)` fact on a **single line**. The expansion machinery in `engine/pddl_problem.py` and `engine/search_problem.py` is line-oriented by design - `_clone_named_lines` and `_clone_search_lines` both duplicate whole lines - so a fact spread across several lines will be cloned incorrectly, and the resulting file will be malformed in a way that only surfaces as a puzzling 'unsolvable' from the planner. The same line-orientation is why `_clone_named_lines` explicitly skips comment lines: an ASCII diagram in a comment could otherwise inject an unmatched parenthesis.

6.3 A known limitation, separate from the bug above

Grounding cost for large grids

ENHSP's grounding step is memory-hungry for large formations. A 16-drone 4x4 grid blew past a 6 GB heap during grounding in local testing with the **default** grounder. This is independent of the defect above and was never caused by it. The mitigation is the `-gro naive` flag described in section 4.5, which avoids the default grounder's combination-building blowup and grounds a 36-drone 6x6 grid in about two seconds with an ordinary default heap. If a large swarm ever returns 'Unsolvable' or runs out of memory again, check the drone count and the grounding time before assuming the PDDL itself is wrong.

7. Summary

- **Only the leader is planned.** ENHSP solves one corridor; every member's track is derived from it afterwards by one `formation_member_route` call per cell.
- **The drone count becomes a shape** through `grid_dimensions`: a perfect square forms a square, anything else the most-square rectangle that tiles it exactly, a prime degenerates to a line. The minimum is `MIN_GRID_DRONES = 4`.
- **One constant drives everything.** `GRID_SPACING_M = 10.0` produces both the descriptive PDDL slot offsets and the actual flown tracks, so unlike the V there is nothing to keep in sync by hand.
- **The domain is generic over the drone count**, because every collective action uses `(forall (?m - drone) (or (not (grid-member ?m)) ...))` rather than naming drones.
- *`grid-closed` is the interlock* that stops the leader flying two legs without its members catching up.
- *`close-up-grid` moves the entire member set in one action*, using a doubly-nested conditional delete - the most advanced PDDL construction in the project.
- *`expand_grid_drones` always rebuilds from scratch*, because a grid's shape changes non-monotonically between counts - 9 drones is a 3x3 square, not '2x2 plus one'.
- **The neighbour mesh is rebuilt, not cloned**, because cloning the template's single leader-to-member pair would wire every distant cell directly to the leader.
- **Each drone coordinates with at most four neighbours** - front, back, left, right, never diagonals - whatever the size of the swarm. That constant coordination burden is the plan's central claim.
- *`-gro naive` is required for large grids*, because ENHSP's default grounder blows up on formation-style domains past about a dozen drones regardless of heap size.
- **The 2026-09-16 defect was a formatting problem**: a fact written across three lines was cloned line-by-line into a valueless fragment. Keep every fact on one line.
- **At runtime there is still no peer link.** The neighbour mesh is a planning-time abstraction; cohesion is achieved by each drone flying its own precomputed parallel track.